A Hopf Algebra on Proof Trees over a Nonmonotonic Consequence Relation

Abstract

We construct a pre-Lie algebra on proof trees for a multisuccedent sequent calculus equipped to a primitive base consequence relation which is assumed to be nonmonotonic. The sequent calculus naturally gives rise to decorated and labelled rooted trees, while the inadmissibility of the Weakening rule gives rise to a constrained grafting operation on trees. We define a proof-theoretically meaningful equivalence relation which identifies proof-trees differing only in inessential or bureaucratic scheduling or reassociation choices. Passing to the induced quotient we show that there is a clean separation between cancelling and genuinely asymmetric contributions to the associator, yielding a pre-Lie identity for proof-tree composition. This extends to a Hopf algebra by the Oudom-Guin construction. This suggests that Hopf-algebraic methods on proof trees are a potential tool for structural proof theory, in the sense that the presence or absence of structural rules controls the existence and form of the algebraic structure.

1 Introduction

The aim of this paper is to define a proof-theoretically meaningful algebraic structure in a setting where monotonicity and familiar structural rules are not built in from the outset. We work in a setting where non-monotonicity is reflected proof-theoretically by the *failure* of Weakening. Structural rules such as weakening and contraction have of course been studied extensively in linear and other substructural logics, where their management gives rise to resource-sensitive or relevance-sensitive proof systems. Our aim here is different: rather than studying the algebraic consequences of controlling structural rules within an otherwise monotone proof system, we seek an algebraic account of proof composition for a consequence relation that is nonmonotonic in the meta-sense — one where provability itself may be defeated by enlarging the premiss set. In this sense, the primitive consequence

relation is meant to reflect patterns of defeasible inference whose validity is not necessarily preserved under addition of new information.

We begin with a primitive consequence relation on a pre-logical vocabulary and use it as the only nonlogical input for a sequent calculus. The resulting derivations are represented by finite rooted proof trees, and proof composition is defined by grafting one proof tree into another at a leaf with matching sequent interface.

Two distinct issues then arise. The first is combinatorial: how do iterated graftings decompose? The second is proof-theoretic: which apparent distinctions between ordered pairs of grafts are genuine and which are merely bureaucratic presentation artefacts? The central contribution of the paper is to show that these two questions can be separated cleanly. The geometry of a graft pair yields a decomposition into independent and nested cases. Passing to a quotient of raw graft-pair data isolates the quotient-trivial overlap sector, leaves a rigid residual sector, and makes the associator comparison transparent.

The argument proceeds in three stages. First, one shows that every graft pair is governed by a simple rooted-tree geometry: the two insertions are either nested or independent. Second, one passes from raw graft-pair witnesses to a quotient that identifies the genuinely bureaucratic symmetries of proof composition. This separates a quotient-trivial noise sector from a residual sector. Third, one proves that the mixed residual case is eliminated, so that the residual contribution classes become rigid enough to be matched across the two bracketings of the associator.

The key point is therefore not that an external tree algebra is imposed on proofs, but that the proof-theoretic structure itself determines the relevant rooted trees, the grafting operation, the quotient of bureaucratic symmetries, and ultimately the class-level associator comparison from which the pre-Lie identity emerges.

Relation to substructural logics. It is worth distinguishing the nonmonotonicity studied here from the related but different phenomenon in linear logic. Linear logic exhibits what one might call *inner* nonmonotonicity: the inference connector $\multimap$ is not monotone, in the sense that provability of $\vdash A \multimap B$ does not imply provability of $\vdash X \otimes A \multimap B$. However, the *provability relation* of linear logic is itself monotone in the meta-sense: adding new axioms or rules cannot destroy existing proofs. The present setting is different: the primitive relation $\mid\!\sim_0$ is allowed to be nonmonotone in the *meta* sense, so that provability itself may be defeated by enlarging the premiss set. This is the stronger, logical notion of nonmonotonicity studied

in the literature on defeasible and nonmonotonic reasoning, and it is what makes the absence of weakening proof-theoretically essential rather than merely convenient.

The structural rules programme. The present paper establishes the base case of a broader research programme. The sequent calculus $NMMS(\mid\!\sim_0)$ is currently formulated with multisets (so exchange is built in) and without weakening or contraction. A natural next question is: which structural rules obstruct the existence of a compatible product and coproduct, and hence of a Hopf algebra on proof trees? Adding weakening would make the sequent interfaces non-rigid, potentially destroying the exactness of the leaf-matching constraint on which the coproduct relies. Adding contraction would identify multisets differing in formula multiplicities, changing the quotient structure in ways that may break the noise/residual separation. This programme — using Hopf-algebraic compatibility as a diagnostic for structural rules — is left for subsequent work, but it provides the conceptual frame within which the present results should be read.

Lean formalization. The main results of this paper have been fully formalized in Lean 4 using Mathlib. The central theorem `classLevel_preLie_identity` has been machine-verified and depends only on the standard foundational axioms `propext`, `choice`, and `Quot.sound`. In particular, no `sorry` placeholders appear in the live proof chain. The formalization comprises approximately 19 000 lines across three main files and served as the primary vehicle through which the quotient structure and the noise/residual separation were discovered; the prose paper is a conservative account of what the formal development established.

The presentation is intentionally more conservative than the internal Lean development. We keep only the mathematically significant layers of the construction in the main text:

$$\begin{aligned}\text{primitive consequence} &\longrightarrow \text{derivability}\\ &\longrightarrow \text{proof trees} \longrightarrow \text{grafting}\\ &\longrightarrow \text{graft-pair quotient} \longrightarrow \text{class-level pre-Lie identity.}\end{aligned}$$

The paper is organized accordingly. Section 2 defines the primitive consequence relation, the calculus $NMMS(\mid\!\sim_0)$, and its induced derivability relation. Section 3 develops proof trees as rooted ordered objects. Section 4 then recalls the algebraic notion of pre-Lie algebra to which the later quotient analysis leads. Sections 5 and 6 develop grafting and the graft-pair decomposition underlying the associator analysis. Sections 7–9 contain the

main quotient argument: bureaucratic equivalence, the separation of noise from residual contribution classes, elimination of mixed residual classes, and the class-level pre-Lie identity. Section 10 records the cut-based coproduct material only as preparatory structure for future work.

2 Primitive consequence and induced derivability

2.1 Base vocabulary and full language

Let L_0 be a set of atomic sentence letters. We set

$$L_B := L_0 \cup \{\bot\},$$

where $\bot$ is treated as a primitive incompatibility marker.

Definition 1. *The full language L is generated inductively from the base fragment L_B by*

$$A ::= p \mid \bot \mid (A \Rightarrow A) \mid (A \wedge A) \mid (A \vee A) \mid \neg A, \qquad p \in L_0.$$

Remark 2. *At the base level, $\bot$ is not governed by any special elimination rule. It functions simply as a primitive marker of incompatibility. In particular, sequents such as $A, B, C \mathrel{|\!\!\sim_0} \bot$, with $A, B, C \in L_0$, may be used to record material incompatibility without imposing any general explosion principle.*

2.2 Primitive consequence

Definition 3. *A* primitive consequence relation *is a binary relation*

$$\mathrel{|\!\!\sim_0} \subseteq \mathit{MSet}(L_B) \times \mathit{MSet}(L_B),$$

where $\mathit{MSet}(L_B)$ denotes the finite multisets of formulas of the base fragment L_B.

Remark 4. *This is the proof-theoretically intended setup: primitive consequence is given only on the pre-logical fragment, and the logical vocabulary is added above it by the sequent rules.*

Remark 5. *No structural conditions are imposed on $\mathrel{|\!\!\sim_0}$. In particular, it need not be monotone: from $\Gamma \mathrel{|\!\!\sim_0} \Theta$ one cannot in general infer either $(\Gamma, A) \mathrel{|\!\!\sim_0} \Theta$ or $\Gamma \mathrel{|\!\!\sim_0} (\Theta, A)$.*

2.3 Sequents

Definition 6. *A* multi-succedent sequent *is an expression*

$$\Gamma \mid\!\sim \Delta,$$

where $\Gamma, \Delta \in MSet(L)$.

We work with multisets in order not to presuppose structural features beyond those explicitly built into the proof system.

2.4 The calculus $NMMS(\mid\!\sim_0)$

We now define the nonmonotonic multi-succedent sequent calculus generated by $\mid\!\sim_0$.

Definition 7. *The derivations of* $NMMS(\mid\!\sim_0)$ *are generated by the following clauses.*

Base axiom. Whenever $\Gamma, \Theta \in MSet(L_B)$ and $\Gamma \mid\!\sim_0 \Theta$, we may form the sequent

$$\frac{\Gamma \mid\!\sim_0 \Theta}{\Gamma \mid\!\sim \Theta} \quad \textit{(Base)}.$$

Implication rules.

$$\frac{\Gamma \mid\!\sim A, \Theta \qquad B, \Gamma \mid\!\sim \Theta}{(A \Rightarrow B), \Gamma \mid\!\sim \Theta} \quad (\Rightarrow L)$$

$$\frac{A, \Gamma \mid\!\sim B, \Theta}{\Gamma \mid\!\sim (A \Rightarrow B), \Theta} \quad (\Rightarrow R)$$

Conjunction rules.

$$\frac{A, B, \Gamma \mid\!\sim \Theta}{(A \wedge B), \Gamma \mid\!\sim \Theta} \quad (\wedge L)$$

$$\frac{\Gamma \mid\!\sim A, \Theta \qquad \Gamma \mid\!\sim B, \Theta}{\Gamma \mid\!\sim (A \wedge B), \Theta} \quad (\wedge R)$$

Disjunction rules.

$$\frac{A, \Gamma \mid\!\sim \Theta \qquad B, \Gamma \mid\!\sim \Theta}{(A \vee B), \Gamma \mid\!\sim \Theta} \quad (\vee L)$$

$$\frac{\Gamma \mid\!\sim A, B, \Theta}{\Gamma \mid\!\sim (A \vee B), \Theta} \quad (\vee R)$$

Negation rules.

$$\frac{\Gamma \mid\!\sim A, \Theta}{\neg A, \Gamma \mid\!\sim \Theta} \quad (\neg L) \qquad \frac{A, \Gamma \mid\!\sim \Theta}{\Gamma \mid\!\sim \neg A, \Theta} \quad (\neg R)$$

Remark 8. *The logical rules are standard active-formula rules. The important structural choice is not the shape of these rules but the absence of built-in structural principles such as weakening. This is the proof-theoretic setting in which the primitive relation $\mid\!\sim_0$ genuinely matters.*

Remark 9. *No weakening or contraction rules are assumed. In particular, the rules*

$$\frac{\Gamma \mid\!\sim \Theta}{\Gamma, A \mid\!\sim \Theta} \qquad \frac{\Gamma \mid\!\sim \Theta}{\Gamma \mid\!\sim \Theta, A}$$

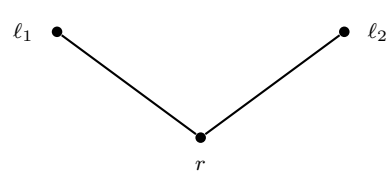
are not built into the system.

Remark 10. *The base relation is extended by logic rather than replaced by logic. In other words, NMMS($\mid\!\sim_0$) is the deductive closure generated from the primitive inferential relation $\mid\!\sim_0$.*

Remark 11. *The next two figures serve different purposes. The first displays a small closed derivation both proof-theoretically and graph-theoretically. The second is a schematic picture of grafting by substitution at a matching leaf.*

$$\dfrac{\dfrac{\Gamma \mid\!\sim_0 (A \Rightarrow B), \Theta}{\Gamma \mid\!\sim (A \Rightarrow B), \Theta}\,\textsc{Base} \qquad \dfrac{\Gamma \mid\!\sim_0 C, \Theta}{\Gamma \mid\!\sim C, \Theta}\,\textsc{Base}}{\Gamma \mid\!\sim ((A \Rightarrow B) \wedge C), \Theta}\,\wedge R$$

The same derivation may also be displayed in graph-theoretic form as an ordinary rooted ordered tree. To match the proof-theoretic orientation above, we place the root at the bottom and draw edges upward toward the premisses.



$$\begin{aligned}
\rho(r) &= \wedge R, & \sigma(r) &= \Gamma \mid\!\sim ((A \Rightarrow B) \wedge C), \Theta, \\
\rho(\ell_1) &= \textsc{Base}, & \sigma(\ell_1) &= \Gamma \mid\!\sim (A \Rightarrow B), \Theta, \\
\rho(\ell_2) &= \textsc{Base}, & \sigma(\ell_2) &= \Gamma \mid\!\sim C, \Theta.
\end{aligned}$$

Formally, this is the same proof object viewed as a finite rooted ordered tree (V, E, r) together with a sequent-labeling map $\sigma : V \to \mathsf{Seq}(L)$ and a partial rule-labeling map ρ defined on the internal vertices. If an internal vertex v has ordered children $v_1, \ldots, v_n$, then the sequents $\sigma(v_1), \ldots, \sigma(v_n)$ are exactly the premisses of an instance of the rule $\rho(v)$ with conclusion $\sigma(v)$. Thus the bottom sequent of an inference is the label of the vertex itself, while the top sequents are recovered from the labels of its children. Leaves corresponding to instances of the primitive base relation may be marked explicitly as Base-vertices.

2.5 Induced derivability

Definition 12. *For $\Gamma, \Delta \in MSet(L)$, we write*

$$\Gamma \vdash_{NMMS(\mid\!\sim_0)} \Delta$$

if there exists a derivation of the sequent $\Gamma \mid\!\sim \Delta$ in the calculus $NMMS(\mid\!\sim_0)$.

Remark 13. *This explicitly separates the primitive relation $\mid\!\sim_0$ from the induced derivability relation $\vdash_{NMMS(\mid\!\sim_0)}$. In a nonmonotonic setting, that distinction is mathematically significant.*

3 Proof trees

3.1 Tree representation

Definition 14. *A* proof tree *is a finite rooted ordered tree whose vertices are labelled by sequents, whose internal vertices are tagged by one of the rule symbols*

$$\mathsf{Base},\ \Rightarrow L,\ \Rightarrow R,\ \wedge L,\ \wedge R,\ \vee L,\ \vee R,\ \neg L,\ \neg R,$$

and whose immediate subtrees record the premises of the inference at that vertex. The sequent at the root is called the conclusion *of the proof tree.*

Remark 15. *The order on immediate subtrees is part of the formal object. This is the only reason addresses are later coded by finite sequences of child indices. Conceptually, the objects are ordinary rooted proof trees.*

Proposition 16. *Every derivation d in $NMMS(\mid\!\sim_0)$ determines a proof tree $Tree(d)$ with the same conclusion sequent.*

Proof. The proof is by induction on the derivation:

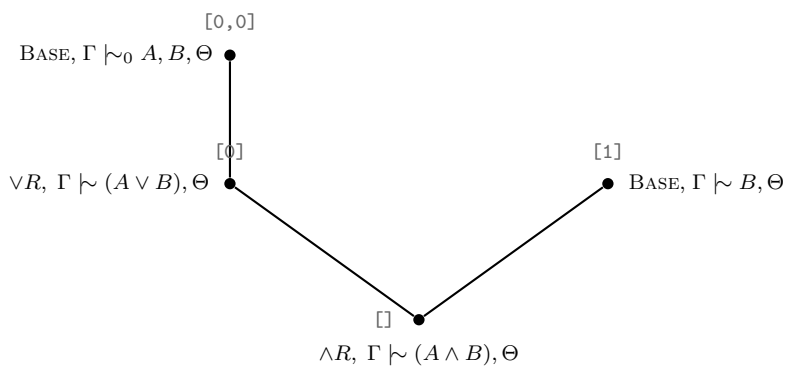
- a base axiom becomes a leaf;
- each logical rule becomes a node whose children are the proof trees of the premises;
- the root sequent is the conclusion of the inference.

□

3.2 Addresses

Definition 17. *An* address *in a proof tree is a finite sequence of natural numbers indicating a path from the root by successive child indices. The empty address selects the whole tree.*

Remark 18. *Addresses are not conceptually basic. They are simply a convenient way of coding vertices and rooted subtrees in the formal development.*



Each node carries a rule tag and a sequent label. Addresses (grey `monospace`) record paths from the root by child index: `[0]` is the first child of the root, `[1]` the second, and `[0,0]` the first child of `[0]`. The nodes at `[0,0]` and `[1]` are leaves (open premiss positions); the node at `[0]` is an internal node with one child.

4 Matching-leaf grafting

4.1 Local grafting

Definition 19. *Let u and t be proof trees. A leaf $a \in Addr(t)$ is* graftable *for u if the sequent labelling that leaf is exactly the conclusion of u.*

Definition 20. *A* matching-leaf grafting *of u into t is the proof tree obtained by replacing a graftable leaf of t by the tree u.*

Definition 21. *We write Graft(u,t) for the finite list of proof trees obtained by matching-leaf grafting of u into t.*

Remark 22. *This operation is controlled by exact interface matching. It does not rely on extending contexts by structural rules.*

Matching-leaf grafting is substitution of one proof tree into a leaf of another carrying the same sequent interface. In the next display, the left tree t contains a base leaf labelled by $\Gamma \mid\!\sim (A \vee B), \Theta$. The middle tree u, shown in red, derives that same sequent from the stronger base sequent $\Gamma \mid\!\sim A, B, \Theta$ by an application of $(\vee R)$. Replacing the matching leaf of t by u yields the grafted derivation on the right.

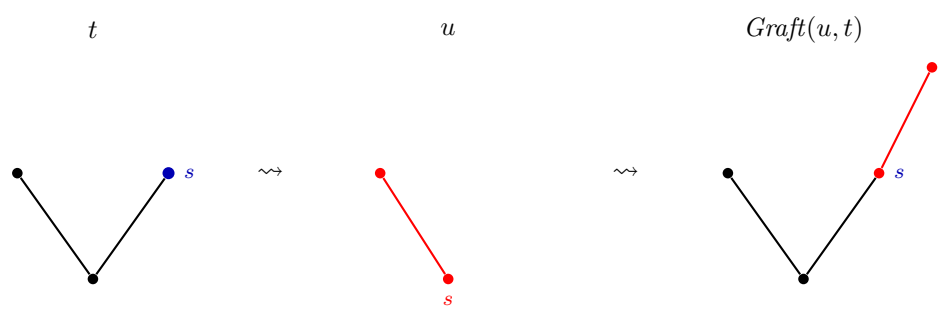
$$\dfrac{\dfrac{\Gamma \mid\!\sim_0 (A \vee B), \Theta}{\Gamma \mid\!\sim (A \vee B), \Theta}\,\textsc{Base} \qquad \dfrac{\Gamma \mid\!\sim_0 C, \Theta}{\Gamma \mid\!\sim C, \Theta}\,\textsc{Base}}{\Gamma \mid\!\sim ((A \vee B) \wedge C), \Theta}\,\wedge R \qquad\qquad \dfrac{\dfrac{\Gamma \mid\!\sim_0 A, B, \Theta}{\Gamma \mid\!\sim A, B, \Theta}\,\textsc{Base}}{\Gamma \mid\!\sim (A \vee B), \Theta}\,\vee R$$

$$t \qquad\qquad\qquad\qquad u$$

$$\dfrac{\dfrac{\dfrac{\Gamma \mid\!\sim_0 A, B, \Theta}{\Gamma \mid\!\sim A, B, \Theta}\,\textsc{Base}}{\Gamma \mid\!\sim (A \vee B), \Theta}\,\vee R \qquad \dfrac{\Gamma \mid\!\sim_0 C, \Theta}{\Gamma \mid\!\sim C, \Theta}\,\textsc{Base}}{\Gamma \mid\!\sim ((A \vee B) \wedge C), \Theta}\,\wedge R$$

$$\textit{Graft}(u,t)$$

The same grafting may also be shown graph-theoretically. The blue leaf of t and the root of u carry the same sequent s, while in *Graft*(u,t) only the inserted top branch is shown in red.



Remark 23. *The Lean formalization is carried out in a slightly more general setting in which the primitive relation is allowed to be defined on multisets of formulas of the full language L. The intended proof-theoretic reading of the present paper is the restricted setup above, and all the formal results apply* a fortiori *to that case.*

4.2 Closure under derivability

Proposition 24 (Closure under derivability)**.** *If $u = \mathit{Tree}(d_{\mathrm{in}})$ and $t = \mathit{Tree}(d_{\mathrm{out}})$ arise from derivations and a is a graftable leaf of t for u, then the grafted tree is again the proof tree of a derivation in $\mathit{NMMS}(\mid\!\sim_0)$.*

Proof. The proof is by induction on the outer derivation. Replacing a premise leaf by another derivation with the same conclusion preserves the validity of the last rule, and hence of the whole derivation. □

Corollary 25. *If u and t are proof trees arising from derivations, then every element of $\mathit{Graft}(u, t)$ also arises from a derivation.*

Proof. Apply the previous proposition to each graftable leaf. □

5 The raw grafting product and graft-pair decomposition

Let $\mathbb{Z}\langle \mathsf{PT} \rangle$ be the free abelian group on proof trees.

Definition 26. *For proof trees u, t, define*

$$u \triangleright t := \sum_{w \in \mathit{Graft}(u,t)} w$$

and extend this bilinearly to $\mathbb{Z}\langle \mathsf{PT} \rangle$.

Remark 27. *This is the proof-tree analogue of Grossman–Larson grafting: one inserts the rooted proof tree u at every matching leaf of t, but only when the sequent interface matches exactly. The operation therefore depends on proof structure, not merely on the underlying combinatorics of rooted trees.*

Proposition 28. *The coefficient of a proof tree w in $u \triangleright t$ is the number of matching-leaf graftings of u into t yielding w.*

Remark 29. *The raw product satisfies the pre-Lie identity as a theorem of the formal development. However, the direct proof of this identity at the raw counting level — by showing that the number of graft-pair witnesses balances on both sides — turns out to be hard to close without additional structure. The quotient analysis developed in Sections 7–9 is not merely an explanation of an identity that holds for simpler reasons; it is the proof strategy that actually works. The equivalence relation on graft-pair codes identifies precisely which witness overcounting cancels and which contributions are genuinely matched across the two bracketings, reducing an opaque counting problem to a typed bijection between pure residual classes.*

5.1 Ordered pairs of grafts

Fix proof trees x, y, z, w. An *ordered pair of grafts from* (x, y, z) *to* w consists of a first graft of y into z at a matching leaf yielding an intermediate tree z', followed by a second graft of x into z' yielding w.

Definition 30. *Such a graft pair is called* outer *if the second graft occurs independently of the first, and* inner *if the second graft occurs inside the subtree introduced by the first. In address terms, these are the incomparable and comparable cases.*

Lemma 31 (Address classification after grafting)**.** *Let y be grafted into z at address a, yielding z'. Let x be successfully grafted into z' at address b. Then either*

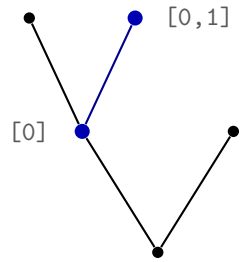
(i) *$b = a +\!\!+ c$ for some non-empty extension c, so the second graft site lies inside the subtree introduced by the first; or*

(ii) *a and b are incomparable, so the two graft sites refer to disjoint parts of the tree.*

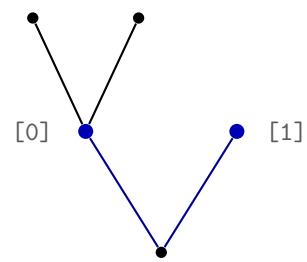
In particular, the case where b is a strict prefix of a — i.e. the second graft would occur at an ancestor of the first graft site — is impossible.

Proof sketch. The impossible case is eliminated by the sequent-interface constraint. If b were a strict prefix of a, then a would extend b, meaning a points into the subtree rooted at address b in z'. But address b in z' is a graftable leaf — a leaf carrying the conclusion of x. No address strictly extending a leaf address can point to a valid subtree, since leaves have no children. This contradiction is the proof-theoretically essential step: it is precisely the rigidity of the leaf-matching constraint, which forbids grafting at a non-leaf position, that eliminates the prefix case. The argument is formalized as `graftMatchingLeafAt_address_classification` in the Lean development. □

Remark 32. *This lemma is the geometric core on which the graft-pair decomposition rests. Its proof depends essentially on the fact that grafting operates on leaves with exact sequent labels, not on arbitrary internal vertices. In a setting with weakening, the leaf-interface constraint would be relaxed and this argument would not go through in the same form. The lemma is therefore sensitive to the structural rule choices made in Section 2.*



comparable
`[0]` is ancestor of `[0,1]`
(inner case)



incomparable
neither is ancestor of the other
(outer case)

Blue marks the pair of graft addresses under comparison. Left: `[0,1]` extends `[0]`, so the second graft site lies *inside* the subtree rooted at the first site — the inner (nested) case. Right: `[0]` and `[1]` are on disjoint branches — the outer (independent) case, where the order of insertion can be reversed.

Theorem 33 (Graft-pair decomposition)**.** *Every ordered pair of grafts is either nested or independent. More precisely, if*

$$y \text{ grafts into } z \text{ to yield } z' \qquad \text{and} \qquad x \text{ grafts into } z' \text{ to yield } w,$$

then exactly one of the following geometric descriptions applies:

(i) Inner case: *the second address extends the first, so the second graft occurs inside the subtree inserted by the first;*

(ii) Outer case: *the two addresses are incomparable, and the two insertions can be performed in the opposite order.*

Proof sketch. This is the content of the full address analysis formalized in the Lean theorem 'two_step_graft_decomposition_full'. If the second address begins with the first, one is in the nested case. Otherwise the addresses are incomparable, and the incomparable-address commutation theorem shows that the order of insertion can be reversed. □

Remark 34. *This theorem is the geometric heart of the associator analysis. It shows that the only essential distinction among graft pairs is between independent and nested insertion.*

Theorem 35 (Commutation of independent graftings)**.** *If two successful graftings occur at incomparable addresses, then performing them in either order yields the same final proof tree.*

Proof sketch. Incomparable addresses refer to disjoint subtrees. Replacing the subtree at one address does not affect the subtree at the other, so the corresponding local modifications commute. □

6 Algebraic preliminaries

We now recall the standard algebraic notion to which the proof-theoretic analysis will later lead.

Definition 36. *A* right pre-Lie algebra *over $\mathbb{Z}$ is an abelian group A equipped with a bilinear product $\triangleright$ satisfying*

$$(x \triangleright y) \triangleright z - x \triangleright (y \triangleright z) = (x \triangleright z) \triangleright y - x \triangleright (z \triangleright y)$$

for all $x, y, z \in A$.

Remark 37. *Every right pre-Lie algebra carries an induced Lie bracket*

$$[x, y] := x \triangleright y - y \triangleright x.$$

The present paper does not yet construct the full enveloping Hopf algebra from this bracket. Instead, it establishes the quotient-controlled proof-theoretic pre-Lie identity that would supply the input for such a construction.

Why quotienting is necessary. One might hope to verify the pre-Lie identity directly at the level of raw proof trees, by tracking explicit address data. This fails for a structural reason: a single term $x \triangleright (y \triangleright z)$ can be witnessed *by many distinct ordered pairs of graft addresses, depending on which matching leaf of $y \triangleright z$ is chosen for the outer step. These witnesses encode the same geometric configuration — the same insertion event at the same relative position in the tree — but differ in the raw list entry that records it. Any equation between graft pairs is therefore really an equation between* equivalence classes *of address records, not between the records themselves.*

More precisely, two witnesses are bureaucratically equivalent *if they agree on which insertion is nested inside which, and whether the outer insertions act on disjoint branches or on ancestor-descendant pairs. The quotient by this equivalence retains exactly the dependency geometry and discards all bookkeeping accidents. The pre-Lie identity*

$$(x \triangleright y) \triangleright z - x \triangleright (y \triangleright z) = (x \triangleright z) \triangleright y - x \triangleright (z \triangleright y)$$

holds in this quotient: the left-hand side and the right-hand side collect the same *noise classes with opposite signs (those cancel) and the same residual classes (those match term for term). Without the quotient, the witness sets on the two sides of the equation are not equal as sets of address records, only equal in cardinality after noise removal; the quotient is what supplies the required bijection.*

7 Quotienting bureaucratic graft-pair data

At raw address level, graft-pair configurations still contain too much presentation data. The next step is to isolate their genuine dependency geometry.

7.1 Graft-pair classes

Definition 38. *A* graft-pair code *(formalized as* `TwoStepCode` *in the Lean development) records a successful ordered pair of grafts together with its geometric type: left outer, right outer, left inner, or right inner. Here "left" and "right" refer to the two bracketings*

$$x \triangleright (y \triangleright z) \qquad \textit{and} \qquad y \triangleright (x \triangleright z),$$

while "outer" and "inner" refer to the independent and nested cases of the graft-pair decomposition.

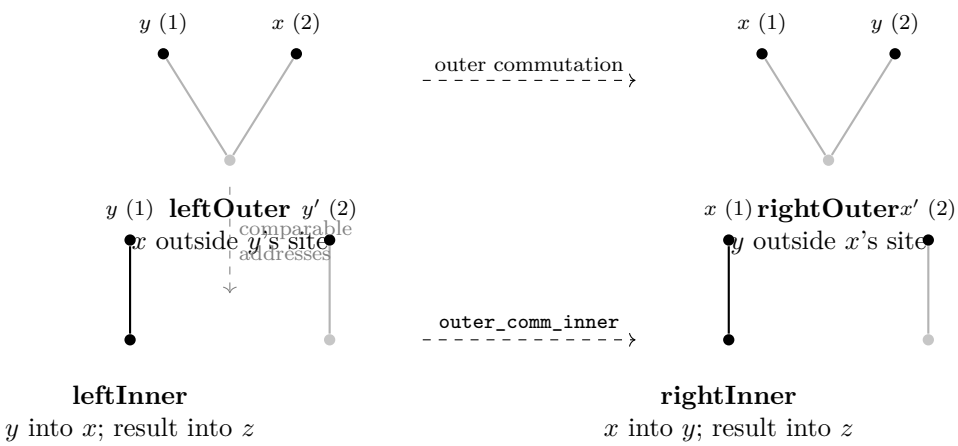
Definition 39. *The* bureaucratic equivalence *on graft-pair codes is the least equivalence relation generated by:*

(i) *commutation of independent outer insertions;*

(ii) *transport of outer left presentations to the corresponding swapped outer presentations;*

(iii) *transport of nested left presentations to the corresponding swapped inner presentations;*

(iv) *the inverse transports from the swapped side.*



The receiver tree z is shown in grey; black nodes are graft sites. Numbered labels (1), (2) indicate the order of the two grafting steps. **Top row** (outer): both insertions act on z or its modification; the two sites are at incomparable addresses. **Bottom row** (inner): step 1 composes two input trees, step 2 places the result into z. The top dashed arrow is outer commutation (leftOuter $\sim$ rightOuter for incomparable addresses). The bottom dashed arrow is the `outer_comm_inner` identification: when graft addresses are *comparable*, a leftOuter code and the corresponding rightInner code record the same proof-composition history viewed from two perspectives.

Remark 40. *Case (ii) above is the* outer commutation*: two independent graftings (at incomparable addresses) can be performed in either order and produce the same final proof tree, so the two derivation histories are identified. This is a scheduling equivalence, analogous to the commutativity of independent rule applications in proof nets, but operating at the level of proof composition rather than within a single proof.*

Case (iii) is more subtle and is the engine of the noise/residual separation proved in Section 8. It identifies a left-outer *code — where x is inserted into z' at an address independent of the address a at which y was first inserted — with a* right-inner *code, in which x is first composed with y and the result is then placed into z. This identification is possible exactly when the two graft addresses are* comparable*: comparability means the second address lies on or above the boundary of the subtree introduced by the first, creating a dependency geometry that admits both readings simultaneously. In the Lean development this case appears as the* `outer_comm_inner` *constructor of* `TwoStepEquiv`*. It is the proof-theoretically distinctive case: in abstract rooted-tree grafting without sequent labels, there is no constraint that forces comparable addresses to interact in this way, and the equivalence would be defined differently or not at all.*

Remark 41. *The bureaucratic equivalence connects to the classical problem of proof identity. The question of when two derivations should count as the same proof — studied in the traditions of proof nets, typed λ-calculus, and categorical proof theory — here receives a specific answer calibrated to proof composition: two graft-pair histories are the same if and only if they are related by the scheduling and dependency equivalences above. The noise sector, defined in Section 8, is precisely the set of classes where this equivalence collapses the distinction between outer and inner bracketings entirely.*

Definition 42. *The quotient set of graft-pair codes is denoted*

$$\mathcal{Q}(x, y, z; w).$$

Inside it we distinguish the left contribution classes

$$\mathcal{C}_{\mathrm{L}}(x, y, z; w)$$

and the swapped-right contribution classes

$$\mathcal{C}_{\mathrm{R}}(x, y, z; w),$$

the latter being associated with the swapped bracketing (y, x, z).

Remark 43. *The quotient is meant to identify proof presentations that differ only by bureaucratic scheduling or by equivalent ways of viewing the same nested composition. This is the formal counterpart of the proof-theoretic idea that identity of proof should ignore inessential order of independent operations.*

7.2 Canonical transport theorems

Theorem 44 (Outer transport)**.** *For every x, y, z, w, outer left contribution classes are canonically in bijection with outer contribution classes on the swapped right side.*

Proof sketch. If the two insertions are independent, they occur at incomparable addresses. The incomparable-address commutation theorem reverses their order without changing the final proof tree. This witness-level transport descends to the graft-pair quotient and yields a bijection of outer classes. □

Theorem 45 (Inner transport)**.** *For every x, y, z, w, inner left contribution classes are canonically in bijection with inner contribution classes on the swapped right side.*

Proof sketch. In the nested case, one uses the inner branch of the graft-pair decomposition: instead of viewing the second graft as occurring inside the previously inserted subtree, one first performs the inner graft inside the inserted tree and then reinserts the result into the ambient proof tree. Again, the transport respects the graft-pair quotient. □

Remark 46. *The outer transport is a strict commutation theorem. The inner transport is a reassociation theorem. Their common role is to show that the quotient keeps the geometric content of the graft pair while discarding the accidental presentation details.*

8 Noise, residual classes, and purity

8.1 Noise and residual sectors

Definition 47. *A left contribution class is called* noise *if the apparent difference between its outer and inner presentations is already quotient-trivial under the comparison with the swapped side. A left contribution class is* residual *if it is not noise.*

Remark 48. *"Noise" does not mean mathematically uninteresting in itself. It means precisely that the class contributes no genuine asymmetry to the associator comparison, because the quotient already identifies the competing presentations.*

Remark 49. *The geometric mechanism producing noise is the following. A left-outer contribution class becomes noise when it simultaneously admits a right-inner presentation. By the* `outer_comm_inner` *identification in the bureaucratic equivalence, this happens exactly when the two graft addresses are comparable. In that case, what appears from the left bracketing as an independent outer insertion of x into z' can equally be read, from the right bracketing, as x being composed into y first and the result placed into z. The class thus contributes equally to both sides of the pre-Lie associator and cancels. This mechanism — comparable addresses producing simultaneously-outer-and-inner classes — is what the formal development calls* left-to-right overlap noise*. The noise sector is therefore not an arbitrary algebraic device but a precise reflection of a geometric degeneracy in the graft addresses.*

Definition 50. *A residual left contribution class is called:*

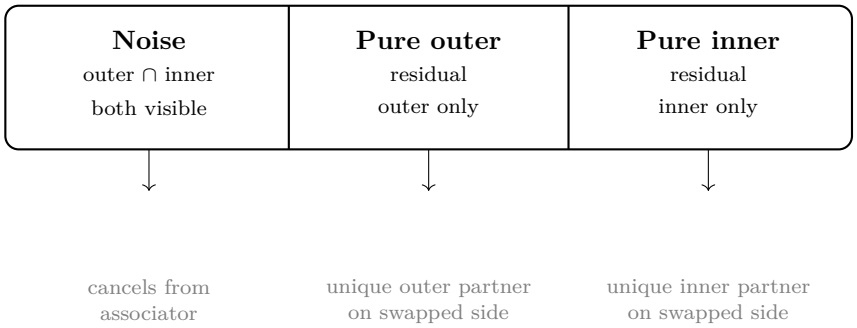
- pure outer *if it has outer support and no inner support;*
- pure inner *if it has inner support and no outer support;*
- mixed *if it has both outer and inner support.*

Theorem 51 (Elimination of mixed residual classes)**.** *There are no mixed residual left contribution classes.*

Proof sketch. Suppose a left contribution class carried both outer and inner support while remaining residual. The geometric analysis of open mixed classes first shows that every outer representative of such a class must lie in a dependent overlap regime. The crucial step is then that a comparable outer witness automatically produces opposite-side inner support. But opposite-side inner support is one of the quotient-trivial overlap mechanisms already isolated by the definition of noise. Thus the class must in fact be noise, contrary to assumption. □

Corollary 52 (Residual purity)**.** *Every residual left contribution class is either pure outer or pure inner.*

Proof. By the theorem, the mixed case is impossible. □



The three sectors partition the left contribution classes at any fixed output w. Noise classes carry both outer and inner support and cancel from the associator by the `outer_comm_inner` identification. The elimination of mixed residual classes (proved above) ensures that the pure outer and pure inner sectors are exhaustive among residual classes. Each residual class then has a unique partner on the swapped right side, yielding the class-level pre-Lie identity.

Remark 53. *This is the decisive structural step in the whole proof. Once mixed residual classes disappear, the residual sector becomes typed and rigid, and the final pre-Lie comparison reduces to separate outer and inner transport theorems.*

8.2 Unique typed partners

Theorem 54 (Unique typed partner theorem)**.** *Let $q \in \mathcal{C}_{\mathrm{L}}(x, y, z; w)$ be residual.*

(i) *If q is pure outer, then there exists a unique pure outer class $q' \in \mathcal{C}_{\mathrm{R}}(x, y, z; w)$ corresponding to q.*

(ii) *If q is pure inner, then there exists a unique pure inner class $q' \in \mathcal{C}_{\mathrm{R}}(x, y, z; w)$ corresponding to q.*

Proof. If q is pure outer, apply the outer transport theorem. If q is pure inner, apply the inner transport theorem. Residual purity guarantees that these cases are disjoint, and the transport maps are equivalences on the relevant typed sectors. □

Theorem 55 (Residual classes seen from the swapped side)**.** *Every residual swapped-right contribution class arises from a pure residual left contribution class.*

Proof sketch. The class-level decomposition on the swapped side yields either a noise class or a residual class. In the residual case, one transports back to the left by the class-level comparison relation and then uses residual purity on the left. □

9 The class-level pre-Lie identity

9.1 Associator contributions at a fixed output

Definition 56. *For proof trees x, y, define the* primitive class-level product support *by*

$$\mathsf{Prod}(x, y) := \{\, w \in \mathsf{PT} : w \in \mathit{Graft}(x, y) \,\}.$$

Remark 57. *This is not yet a quotient object. It is simply the set of outputs occurring in one-step grafting. The quotient enters only in the analysis of how these outputs receive graft-pair associator contributions.*

Definition 58. *Fix x, y, z, w. The* class-level associator at output *w is the assertion that left contribution classes to w decompose into noise, pure outer residual classes, and pure inner residual classes, and that these three sectors compare with the swapped side exactly as described by the noise, outer-transport, and inner-transport theorems.*

Theorem 59 (Class-level associator shape)**.** *For every x, y, z, w, the class-level associator at output w holds.*

Proof sketch. Every left contribution class is either noise or residual. By residual purity, every residual class is pure outer or pure inner. Noise is irrelevant to the associator comparison by definition; pure outer residual classes have unique outer partners by the outer transport theorem; pure inner residual classes have unique inner partners by the inner transport theorem. Finally, every non-noise swapped-right class comes from a pure residual left class. □

Theorem 60 (Class-level pre-Lie comparison)**.** *Fix x, y, z, w. Every left contribution class to the coefficient of w in the associator belongs to exactly one of the following three sectors:*

(i) *quotient-trivial noise;*

(ii) *pure outer residual, with a unique swapped pure outer partner;*

(iii) *pure inner residual, with a unique swapped pure inner partner.*

Proof. This is the operational form of the class-level associator shape. □

9.2 The main theorem

Theorem 61 (Class-level pre-Lie identity)**.** *For all proof trees x, y, z, the class-level associator is symmetric under interchange of x and y. Equivalently, for every output tree w, the contribution classes to*

$$(x \triangleright y) \triangleright z - x \triangleright (y \triangleright z)$$

are matched, after quotienting by bureaucratic equivalence, with the contribution classes to

$$(y \triangleright x) \triangleright z - y \triangleright (x \triangleright z).$$

Proof. Fix w. By the class-level pre-Lie comparison theorem, every left contribution class lies in exactly one of three sectors.

Noise. Noise classes are quotient-trivial by construction and contribute no genuine asymmetry.

Pure outer residual classes. These are matched uniquely across the two bracketings by outer transport.

Pure inner residual classes. These are matched uniquely across the two bracketings by inner transport.

Since these three sectors exhaust the left contribution classes at output w, the class-level associator at w is symmetric under swapping x and y. As w was arbitrary, the class-level pre-Lie identity follows. □

Remark 62. *This theorem is the precise endpoint currently established by the formal development. It is a statement about quotient-invariant associator comparison at the level of contribution classes. It does not yet claim a fully elaborated Hopf algebra in the main body of the paper.*

10 Preparatory coproduct data and Hopf outlook

The formal development also defines a cut-based coproduct datum on proof trees. We record only the part that is presently used with mathematical confidence in the current project.

Definition 63. *An* admissible cut *of a proof tree is a finite antichain of addresses whose associated cut decomposition respects the base-interface constraints tracked in the formal development. The cut determines a pair consisting of a forest of extracted subtrees and a remainder tree.*

Proposition 64. *Every proof tree admits at least one cut decomposition.*

Proof. The empty cut is always admissible, and it yields the trivial decomposition into the empty forest together with the original tree. □

Remark 65. *This coproduct material is preparatory rather than central in the present paper. It shows that the proof-tree side already carries the decomposition data expected of a Connes–Kreimer style coproduct, but we do not yet prove the full compatibility theorems needed for a Hopf algebra in the body of the paper.*

Remark 66. *There are two natural routes from the present work to Hopf algebra.*

(i) *A direct cut/grafting route, in which one relates the cut-based coproduct to proof-tree grafting itself.*

(ii) *An Oudom–Guin route, in which the class-level pre-Lie structure proved here is first extended to a symmetric algebra and only then combined with a primitive coproduct.*

The second route is the one most strongly suggested by the present formal results, and it is the route we have in mind when the abstract mentions Hopf algebraic consequences.

Remark 67 (Current state of the Oudom–Guin development)**.** *The formal development already contains preparatory work toward route (ii). The file* `GrossmanLarssonOudomGuin.lean` *establishes, without any* `sorry`*, the following:*

- *a graft-compatible weight function on proof trees, grading every one-step product output by* $\mathsf{weight}(w) = \mathsf{weight}(x) + \mathsf{weight}(y)$ *whenever* $w \in \mathit{Graft}(x, y)$*;*
- *a proof that any bilinear product on the linear proof-tree carrier whose generator-level behaviour matches the quotient pre-Lie law automatically satisfies the right pre-Lie axiom on the entire carrier (*`generatorRightPreLieLaw_extends`*);*
- *a proof that any such product respects the weight grading on all homogeneous elements (*`generatorWeightLaw_extends`*).*

The remaining step for route (ii) is to construct the specific bilinear product from the quotient data — i.e. to show that the grafting operation descends to a well-defined map on the linear carrier that is compatible with the quotient — and then to apply the Oudom–Guin theorem to obtain the enveloping Hopf algebra on the symmetric algebra. The infrastructure is in place; this is the content of the next paper in this series.

Remark 68 (Why the Connes–Kreimer route is not taken)**.** *The direct cut/grafting route (i) would require the algebraic product to be the forest product (disjoint union of proof trees). However, a forest of proof trees has no natural proof-theoretic interpretation: there is no sequent whose derivation* is *a side-by-side collection of independent proof trees. Proof composition is sequential — one proof's conclusion becomes another's premiss leaf — not parallel. The grafting product is proof-theoretically meaningful precisely because it is proof composition. Route (i) is therefore not pursued here, though a comparison with Connes–Kreimer-style Hopf algebras by forgetting the sequent decoration remains an interesting open problem (see the following remark).*

Remark 69. *A further comparison problem, not treated here, is to relate the present structure to familiar rooted-tree Hopf algebras by forgetting some of the proof-theoretic decoration while retaining the underlying tree and insertion geometry. Such comparison theorems would strengthen the claim that the present construction refines, rather than competes with, established combinatorial Hopf-algebraic frameworks. The key question is whether the Hopf algebra obtained via the Oudom–Guin route depends essentially on the primitive relation $\mid\!\sim_0$ — in the sense that different choices of $\mid\!\sim_0$ yield genuinely non-isomorphic algebraic structures — or whether the structure is universal and the logical content enters only through the choice of basis. Settling this would determine whether the construction encodes something specifically about nonmonotonic proof theory or merely recovers a combinatorial object by a proof-theoretic route.*

Remark 70 (Structural rules as future parameters)**.** *As noted in the introduction, the present paper fixes multiset contexts and omits weakening and contraction. Each structural rule is a potential parameter. Adding weakening makes sequent interfaces non-rigid: one could graft into any leaf by first weakening the context to match. This would change the leaf-matching constraint fundamentally and is expected to obstruct the coproduct compatibility rather than the pre-Lie structure. Adding contraction collapses multisets differing in formula multiplicities, changing which sequents are identified and potentially breaking the noise/residual separation whose geometry depends on the counting of matching leaves. Moving from multisets to sequences (removing the built-in exchange) would add ordered structure to sequents and make the grafting product sensitive to context ordering in ways that require separate analysis. A systematic study of which structural rules are obstructions to Hopf-algebraic compatibility is left for future work.*

11 Conclusion

We have constructed a proof-theoretic route from a primitive consequence relation to a class-level pre-Lie identity on proof-tree composition. Formally, the route is

$$\begin{aligned} \mid\!\sim_0 &\longrightarrow \mathit{NMMS}(\mid\!\sim_0) \\ &\longrightarrow \text{proof trees} \longrightarrow \text{grafting geometry} \\ &\longrightarrow \text{graft-pair quotient} \longrightarrow \text{class-level pre-Lie identity.} \end{aligned}$$

The distinctive point is not merely that rooted trees appear, but that they appear as proof objects carrying exact sequent interfaces. Grafting is therefore proof composition, not arbitrary combinatorial insertion. The graft-pair quotient then isolates what should count as invariant in proof composition: independent commutations and quotient-trivial overlap disappear as bureaucracy, while the residual sector becomes pure and rigid enough to satisfy the class-level pre-Lie identity.

From the standpoint of logic, this means that a nonmonotonic proof theory can carry nontrivial algebraic structure without first collapsing to a monotone or structurally saturated setting. From the standpoint of algebra, it shows that pre-Lie behavior can emerge from proof composition once identity of proof is handled carefully at the level of graft-pair interactions. The Hopf-theoretic extensions suggested by the present development are important, but they are best seen as a next step built on top of the logic result already proved here: after quotienting bureaucratic symmetries, proof composition on these nonmonotonic proof trees satisfies a class-level pre-Lie identity.

The quotient is not a stylistic choice. The direct approach to the pre-Lie identity — proving that the raw witness counts balance on both sides of the associator — was attempted and did not close. The equivalence relation on graft-pair codes, and in particular the `outer_comm_inner` identification of comparable-address graftings, is the structure that makes the proof work. This identification has a proof-theoretic reading: it says that two derivation histories for the same output proof tree, which look geometrically different at the address level, should count as the same composition when the relevant graft sites stand in a dependency relation. The noise/residual separation is then a precise account of which histories are genuinely the same (noise, canceling from the associator) and which are genuinely distinct (residual, matched uniquely across the two bracketings).

The central result is fully machine-verified in Lean 4. The class-level pre-Lie identity depends only on the standard axioms of Lean's foundational system. The broader programme — determining which structural rules obstruct Hopf-algebraic compatibility, and whether the resulting structure encodes specific properties of the primitive consequence relation $\mid\!\sim_0$ — remains open and will be pursued in subsequent work.